23AID302 — BIG DATA ANALYTICS

Final Project Report

A Scalable Big Data Platform for Real-Time Urban Air Quality Tracking and Public Safety Alerting

Group A14

Team member	Register number
Sangeeth S	CB.AI.U4AID24050
Jaganath R	CB.AI.U4AID24021
Niranjan S	CB.AI.U4AID24035
PL Kowshik	CB.AI.U4AID24036

Faculty: Sreeja B P

Date: October 2026

GitHub: https://github.com/jaganath010/A-Scalable-Big-Data-Platform-for-Real-Time-Urban-Air-Quality-Tracking-and-Public-Safety-Alerting

Website: https://jaganath010.github.io/A-Scalable-Big-Data-Platform-for-Real-Time-Urban-Air-Quality-Tracking-and-Public-Safety-Alerting/

Abstract

Urban air pollution is a public-health problem that needs low-latency analytics rather than overnight batch reports. Published air-quality models predict accurately but run on a single machine and stop at prediction: none turns a forecast into an automated public warning. This project builds the distributed platform they lack and runs it end to end.

A 1-NameNode / 4-DataNode HDFS cluster with YARN and Spark 3.5 holds a 154,583,979-row, 33 GB lakehouse that unifies structured CPCB station data (flat CSV) with semi-structured CAMS data (nested JSON). Scala jobs clean and impute the data, engineer 86 features and score a live feed; a PySpark MLlib Gradient-Boosted-Trees model (300 trees) forecasts each city's $PM_{2.5}$ one hour ahead. On 2,074,569 real test rows it reaches $R^2 = 0.8910$ against 0.8587 for the persistence baseline, a 12.18 % reduction in RMSE.

Spark Structured Streaming scores live data in 10-second micro-batches and an alert engine converts the forecast to a CPCB AQI and raises a warning when it exceeds 150. In a live run covering 1–15 October 2026 (82,800 rows, 230 cities) the system raised 27,543 warnings across 221 cities. Results are served through Hive and a public warning board, and delivered by a de-duplicated alert loop (Telegram / e-mail). The report documents the design, the measured results, the validation of a dataset that is 88.9 % synthetic, eleven defects found by running the code, and the evidence for each claim.

Keywords: Hadoop, HDFS, YARN, Apache Spark, Scala, Structured Streaming, Gradient Boosted Trees, Hive, air quality, AQI, alerting.

Contents

Abstract 2
1. Introduction 5
1.1 Problem statement 5
1.2 Objectives 5
1.3 Why big data is needed 5
2. Literature Survey and Gap 6
3. Dataset 7
3.1 The three data layers 7
3.2 Key attributes 7
3.3 The CAMS coverage finding 7
3.4 Synthetic expansion and its validation (veracity) 8
3.5 Live feed 8
3.6 The five V's demonstrated 8
4. System Architecture 9
5. Implementation 10
5.1 HDFS cluster 10
5.2 YARN and Spark 10
5.3 Preprocessing (CleanSelect.scala) 11
5.4 Spark workflow: DAG, partitioning and caching 13
5.5 Feature engineering (FeatureExtraction.scala) 14
5.6 The forecasting model 14
5.7 Live enrichment (LiveEnrichment.scala) 15
5.8 Structured Streaming speed layer (AirQualityStream.scala) 15
5.9 Alert engine and delivery 15
5.10 Hive serving layer 16
5.11 Public warning board 16
6. Results 17
6.1 Forecast accuracy 17
6.2 Model comparison and tuning 17
6.3 What the model learned 18
6.4 Streaming performance 18
6.5 October 2026 live run 18
6.6 Seasonality 19
6.7 Business insight 19
7. Validation and Testing 20
7.1 Defects found by running the code 20
7.2 Independent cross-validation against CAMS 20

8. Evidence 21
9. Challenges and Limitations 22
9.1 Limitations 22
10. Conclusion and Future Work 23
10.1 Future work 23
11. Repository and How to Reproduce 24
11.1 Running it 24
References 25

1. Introduction

1.1 Problem statement

Three gaps separate what exists from what a city needs. First, scale: existing air-quality models run on a single machine, while a city-wide, multi-year deployment produces hundreds of millions of readings that no single machine can hold in memory. Second, latency: batch analytics report after the fact, but a pollution spike is a health event that is happening now and a warning delivered tomorrow has no value. Third, automation: published frameworks stop at prediction and do not close the loop by turning a forecast into an actionable public-safety alert.

1.2 Objectives

#	Objective	Success criterion	Final result
1	Fault-tolerant HDFS cluster	≥ 2 DataNodes, 3× replication, 0 corrupt blocks	4 DataNodes; fsck HEALTHY, 0 missing / corrupt
2	Distributed compute on the cluster	Spark running under YARN	Jobs ran as YARN applications on 4 NodeManagers
3	Ingest into distributed storage	Split into 128 MB blocks	232 raw blocks, 696 replicas; all layers 33 GB
4	Preprocessing in Scala	Reads HDFS, cleans, writes back to HDFS	146,276,379 rows → 10.0 GB Parquet, 241 partitions
5	Spatial-temporal features	T+1 target, rolling history, CPCB AQI	93,216,105 labelled rows, 86-dim vector
6	Distributed GBT forecaster	Must beat the persistence baseline	R^2 0.8910 vs 0.8587, RMSE − 12.18 %
7	Streaming speed layer	Structured Streaming micro-batches	10 s trigger, ≈ 7 s latency per batch
8	Close the loop with alerts	Warning when forecast AQI > 150	27,543 warnings, 221 cities (Oct 2026)

1.3 Why big data is needed

The dataset is 154.6 million rows and 33 GB, on a development host with 15.7 GB of RAM: it fits only if it is partitioned and processed in a distributed way. The live feed must be scored every ten seconds. The sources come in two data models (flat CSV and nested JSON). And 88.9 % of the structured rows are synthetic expansion, so the truthfulness of the data has to be established by computation rather than assumed. These correspond to the five V's, summarised in the table below and demonstrated in Section 3.6.

Table 1. The five V's in this project

V	Meaning here	Measured evidence
Volume	Too large for one machine	154,583,979 rows, 33 GB, 232 blocks per raw copy, 4 DataNodes
Variety	Structured + semi-structured	Flat CSV and nested JSON on one 17-column schema, read by native Spark readers
Velocity	Live data, low latency	10 s micro-batches, ≈ 7 s latency; hourly alert loop
Veracity	Can the data be trusted?	5 paired tests, CPCB-vs-CAMS cross-check, 11 defects fixed, every imputed value flagged

V	Meaning here	Measured evidence
Value	Does it help people?	12.18 % lower error; 27,543 warnings issued one hour ahead with health advice

2. Literature Survey and Gap

The primary base paper is “Air Quality Prediction using Multi-Source Remote Sensing Data Integration with Hybrid Deep Learning Framework (MAST-Net)”, published in Nature Scientific Reports in December 2025. Its contribution is that fusing several environmental data streams — remote sensing, meteorology and ground sensors — improves accuracy over single-source models.

The gap this project addresses is architectural rather than statistical. The framework is built on a localised, single-machine design with no distributed infrastructure for high-volume, multi-city scaling and no streaming mechanism for immediate warnings. This project ports the multi-source fusion idea onto Hadoop and Spark (CPCB stations, CAMS reanalysis and live weather), adds Structured Streaming for latency, and closes the loop with an alert engine that turns a forecast into a warning message.

Supporting foundations are the Resilient Distributed Dataset abstraction (Zaharia et al., NSDI 2012), the Hadoop Distributed File System (Shvachko et al., MSST 2010) and Friedman's gradient boosting machine (Annals of Statistics, 2001), on which the forecaster is based.

3. Dataset

3.1 The three data layers

The project combines three layers, each with one job. The structured layer trains the model because it is the only one with the label and thirteen years of seasonality. The semi-structured CAMS history is an independent source on the same cities and hours, used to validate the data. The live CAMS and weather feed is what makes alerting possible.

Table 2. Data layers

Layer	Source	Format	Rows	Size	Period
Structured	CPCB stations (+ validated synthetic expansion)	Flat CSV	146,276,379	27.5 GB	2010–2023
Semi-structured	CAMS reanalysis via Open-Meteo (history)	Nested JSON	8,263,440	458 MB	2022-08 → 2026-09
Semi-structured	CAMS + weather, live (keyless, CC-BY 4.0)	Nested JSON	82,800 per refresh	≈ 20 MB	1–15 Oct 2026

No API key is needed anywhere: Open-Meteo serves CAMS data and weather without authentication. The unified table (next table) has one 17-column schema and is partitioned by source format.

Table 3. Rows by data source in the unified table (distributed count on YARN)

data_source	format	rows	cities
synthetic	structured_csv	130,023,448	241
real	structured_csv	16,252,931	241
cams_history	semistructured_json	8,263,440	230
live_api	semistructured_json	44,160	230
TOTAL		**154,583,979**	

3.2 Key attributes

- Identity and time: station_id, city, state, hourly timestamp.
- Pollutants: pm25, pm10, no2, so2, co, o3, nh3 — the seven pollutants of the CPCB National AQI. They give the AQI and the label (next-hour $PM_{2.5}$).
- Weather: temperature, humidity, wind speed and direction, pressure, rainfall, solar radiation.
- Context: traffic_index, urbanization_index, climate_type, is_rush_hour.
- Provenance: data_source, source_format and one *_was_imputed flag per imputed column, so measured and estimated values are never confused.

After cleaning the table has 44 columns; feature engineering produces 54 numeric features and a 32-wide one-hot encoding, a vector of 86 dimensions.

3.3 The CAMS coverage finding

The brief asked for semi-structured data covering 2010–2023. This was tested before anything was built. The source rejects requests before 2013-01-01, and for 2013-01 through 2022-07 it returns timestamps with every value null (0 % fill). Real data begins in August 2022 (89.7 % fill) and is about

99.8 % filled from September 2022 (ammonia is 0 % throughout). Delivering 2010–2023 would have produced ten years of rows that look like coverage and contain nothing, so coverage starts in August 2022 and the reason is stated. 230 of 241 cities resolved to coordinates; 11 small localities had no geocode and were excluded rather than given invented coordinates.

3.4 Synthetic expansion and its validation (veracity)

88.9 % of the structured rows (130,023,448 of 146,276,379) are a synthetic expansion of 16,252,931 real station-hours. Because accuracy claims would be meaningless if the synthetic rows were easier to predict than real ones, every synthetic station is tested against its own real parent with five paired tests, and a FAIL blocks the run.

Table 4. The five paired veracity tests

#	Test	The failure it catches
1	Persistence R^2 (synthetic vs real parent)	Fake data that is easier to predict than real data
2	Autocorrelation at lags 1–6 h	Random noise or over-smoothing
3	Quantiles p5–p95	Drift and inflated variance
4	Missingness rate	Gaps filled in to make the data look easy
5	Cross-correlation between pollutants	Noise that breaks physical links between pollutants

The first generator scored 0.856 persistence R^2 on synthetic data against 0.733 on real data on test 1: it would have failed, and was replaced. The final generator passes all five tests. Independently, the CPCB station data was compared with CAMS on 1,152,707 matched city-hours (Section 7.2).

3.5 Live feed

The live feed is CAMS air quality and Open-Meteo weather for 230 cities, fetched in the same cycle so that their time windows align (an earlier mismatch joined to zero rows). Raw CAMS JSON carries only 28 of the model's 54 numeric features, so a Scala job (LiveEnrichment) adds weather, derived features, city profiles and provenance flags (Section 5.7).

3.6 The five V's demonstrated

Volume: 154.6 M rows and 33 GB, replicated three times across four DataNodes. Variety: the Spark readers ingest flat CSV and nested JSON natively and union them on one schema in a single job. Velocity: the streaming job processes micro-batches every ten seconds. Veracity: five paired tests, a cross-check against an independent source, eleven defects found and fixed, and a provenance flag on every filled value — the model even ranks the pm25_was_imputed flag sixth of 86 features, evidence that it learned to discount estimated values. Value: a forecast 12.18 % more accurate than the honest baseline, converted into warnings delivered an hour ahead.

4. System Architecture

The system is Lambda-style: a batch layer trains the model on thirteen years of history, and a speed layer scores live data against the same model, both sharing one HDFS lakehouse. Figure 1 shows the components; the model is stored in HDFS and loaded by the streaming job.

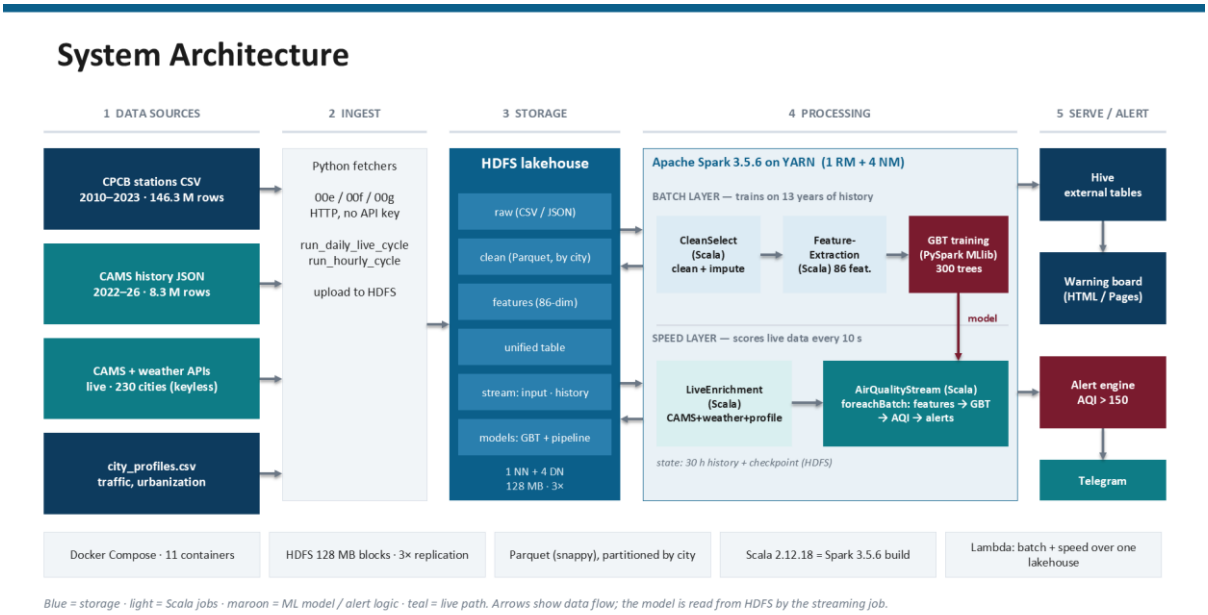


Figure 1. System architecture: sources, ingest, HDFS storage, Spark on YARN (batch and speed layers), serving and alerting

Table 5. Components and their roles

Component	Technology	Role
Storage	Hadoop HDFS 3.3.6 (1 NameNode + 4 DataNodes)	128 MB blocks, 3× replication (derived layers 2×), Parquet + Snappy
Resource management	Hadoop YARN (1 ResourceManager + 4 NodeManagers)	Schedules Spark applications; each NodeManager 3 GB, 2 vcores
Batch compute	Apache Spark 3.5.6, Scala 2.12.18	CleanSelect (clean, impute) and FeatureExtraction (86 features, label)
Machine learning	PySpark MLlib GBTRegressor	Trains and evaluates the forecaster; model saved to HDFS
Speed layer	Spark Structured Streaming (Scala)	LiveEnrichment and AirQualityStream: 10 s micro-batches, alert engine
Serving	Apache Hive (external tables), HiveServer2	SQL over the Parquet lake; export to the dashboard
Alerting	Python dispatcher (Telegram / e-mail)	De-duplicated delivery of warnings
Deployment	Docker Compose (11 containers)	Cluster runs on one development host

Scala 2.12.18 is not a free choice: it is the version Spark 3.5.6 is built against, and a 2.13 or 3.x build compiles but fails at run time with NoSuchMethodError. Scala is used for the latency-critical and cluster-side jobs; PySpark is used for model training and analytics.

5. Implementation

5.1 HDFS cluster

A cluster of one NameNode and four DataNodes runs in Docker. Four DataNodes, not the minimum two or three, because with three nodes and replication 3 every node must hold every block, so nothing distributes and a failed node has nowhere to re-replicate. The raw 27.5 GB dataset is split into 232 blocks of at most 128 MB (the number of blocks per file is ceil(filesize / 128 MB)), giving 696 replicas and 82.4 GB on disk. fsck reports the filesystem HEALTHY with no missing, corrupt or under-replicated blocks.

```
C:\Users\jagan>docker exec aq-namenode hdfs dfsadmin -report | findstr /C:"Live datanodes" /C:"Hostname" /C:"DFS Used:"
DFS Used: 110869192704 (103.25 GB)
Live datanodes (4):
Hostname: datanode1
DFS Used: 26445000704 (24.63 GB)
Non DFS Used: 91068289024 (84.81 GB)
Hostname: datanode4
DFS Used: 28789288960 (26.81 GB)
Non DFS Used: 88724000768 (82.63 GB)
Hostname: datanode2
DFS Used: 27764670464 (25.86 GB)
Non DFS Used: 89748619264 (83.58 GB)
Hostname: datanode3
DFS Used: 27870232576 (25.96 GB)
Non DFS Used: 89643057152 (83.49 GB)
```

Figure 2. dfsadmin report: 4 live DataNodes

```
C:\Users\jagan>docker exec aq-namenode hdfs fsck /user/hadoop/lakehouse/raw
Connecting to namenode via http://namenode:9870/fsck?ugi=root&path=%2Fuser%2Fhadoop%2Flakehouse%2Fraw
FSCK started by root (auth:SIMPLE) from /172.18.0.2 for path /user/hadoop/lakehouse/raw at Fri Aug 14 04:43:08 UTC 2026


Status: HEALTHY
 Number of data-nodes:  4
 Number of racks:               1
 Total dirs:                    1
 Total symlinks:                0

Replicated Blocks:
 Total size:    29502891370 B
 Total files:   23
 Total blocks (validated):      232 (avg. block size 127167635 B)
 Minimally replicated blocks:   232 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    3
 Average block replication:     3.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0 (0.0 %)
 Blocks queued for replication: 0

Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
 Missing block groups:          0
 Corrupt block groups:          0
 Missing internal blocks:       0
 Blocks queued for replication: 0
FSCK ended at Fri Aug 14 04:43:08 UTC 2026 in 12 milliseconds


The filesystem under path '/user/hadoop/lakehouse/raw' is HEALTHY
```

Figure 3. fsck of the raw layer: HEALTHY, 232 blocks, replication 3.0, 0 missing / corrupt

5.2 YARN and Spark

Spark runs on YARN with four NodeManagers (3 GB and 2 vcores each; maximum container 2,560 MB). A distributed count over the unified table was run as a YARN application and reproduced the known total of 154,583,979 rows. Phase 4 ran in Spark local mode because four 3 GB NodeManagers advertise 12 GB on a 15.7 GB host and YARN executors were killed (exit 137) under memory pressure; YARN execution is demonstrated by the live-enrichment and recount applications.

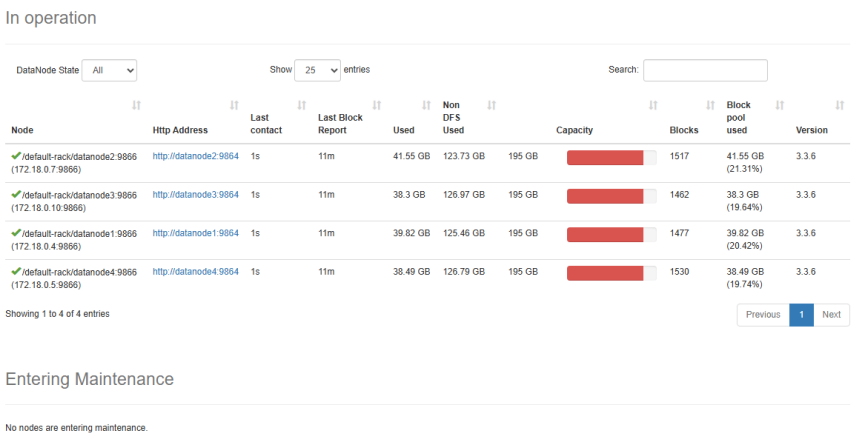


Figure 4. HDFS NameNode UI: DataNodes in service

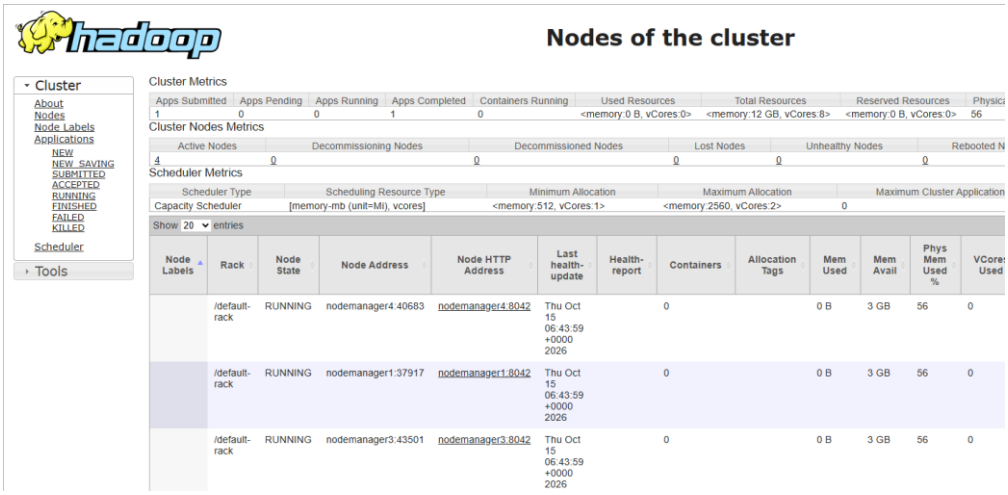


Figure 5. YARN ResourceManager: four NodeManagers, all RUNNING

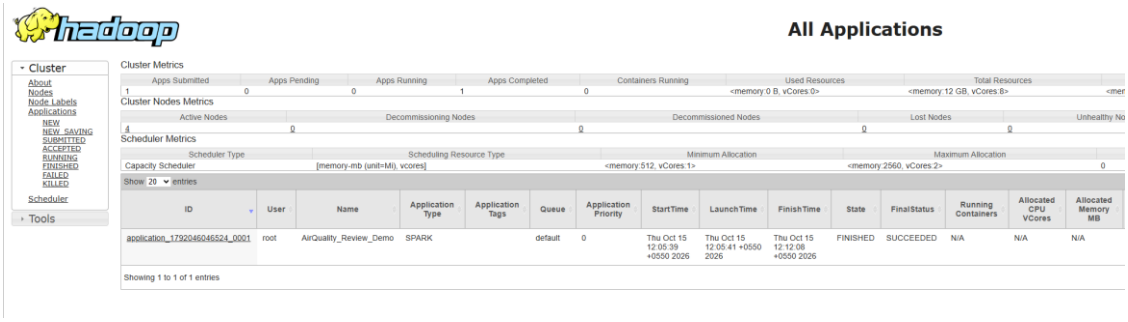


Figure 6. YARN applications: the Spark recount job, FINISHED / SUCCEEDED

5.3 Preprocessing (CleanSelect.scala)

CleanSelect reads the raw CSV from HDFS, applies ten operations and writes snappy-compressed Parquet partitioned by city. It ran on all 146,276,379 rows in about 28 minutes and shrank the data 2.75 times (27.5 GB to 10.0 GB).

1. Explicit 32-column schema, no inference.
2. Type casting; unparseable values become null.
3. Timestamp parsing; rows with unparseable timestamps are dropped.
4. Binary flags repaired to strictly 0 / 1.
5. Physical range clamping (for example humidity to 0–100).

6. Anomaly removal: readings outside sensor-plausible limits are nulled.
7. Three-tier median imputation: station/month/hour, then city/month, then global.
8. Provenance flags: one *_was_imputed column per imputed column.
9. Label protection: pm25_measured keeps the pre-imputation value so the model is never trained on an imputed label.
10. Columnar write: Snappy Parquet, partitioned by city (241 directories).

Only about 46 % of rows carry all seven pollutants, so imputation is necessary; every filled value is flagged.

Table 6. Share of values imputed per column

Column	Imputed	Measured	Column	Imputed	Measured
pm25	27.17 %	72.83 %	nh3	41.72 %	58.28 %
pm10	34.20 %	65.80 %	solar_radiation	40.76 %	59.24 %
no2	21.58 %	78.42 %	pressure	55.28 %	44.72 %
so2	25.50 %	74.50 %	wd_degree	28.77 %	71.23 %
co	25.28 %	74.72 %	o3	26.89 %	73.11 %

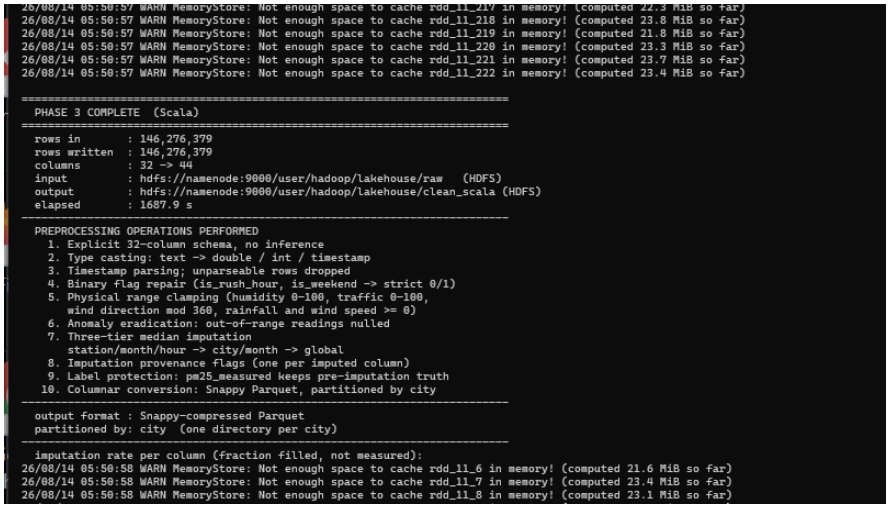


Figure 7. CleanSelect at full scale: 146,276,379 rows in, 146,276,379 out

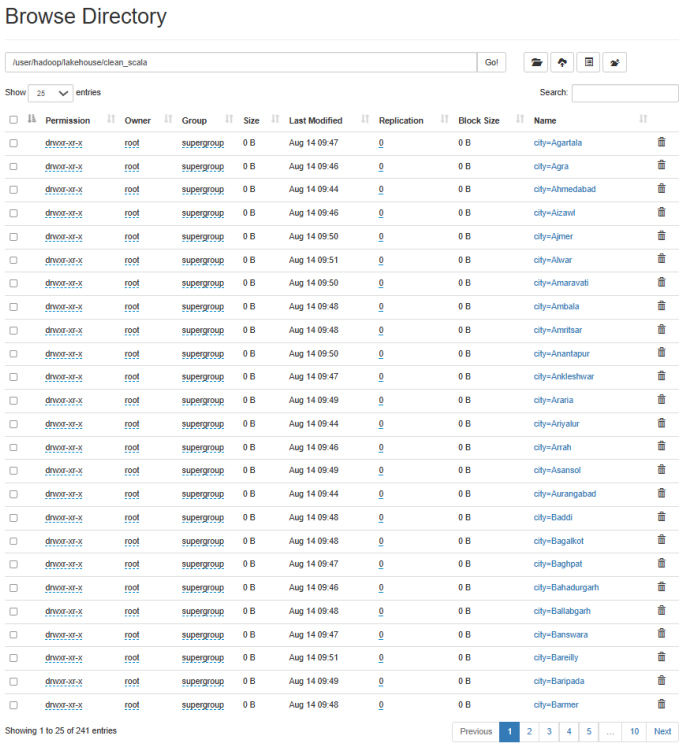


Figure 8. Processed layer in HDFS: one directory per city (241)

5.4 Spark workflow: DAG, partitioning and caching

The jobs use the DataFrame API; Catalyst compiles it to RDD stages, and MLlib's GBT works on RDDs internally (training issued over 1,300 Spark jobs). The feature-engineering job is a chain of six steps: read Parquet (232 parallel tasks, one per block), remove duplicates (a shuffle by station and hour), add rolling-history features (a window shuffle), attach the next-hour label, assemble the 86-dimensional vector, and write one directory per city (a shuffle by city). Rows go 146.3 M → 128.7 M → 93.2 M and the job takes 73.5 minutes.

Table 7. Partitioning and caching choices

Topic	Choice	Reason
HDFS blocks	128 MB blocks	232 input splits = 232 parallel tasks for the raw copy
Disk partitioning	partitionBy(city) = 241 directories	A query for one city reads one directory. Partitioning by year/month produced 5,237 tiny files (244 KB median) and was rejected
Unified table	partitionBy(source_format) = 2	Prunes the structured or semi-structured side
Shuffle partitions	400 / 200 / 120 / 64 / 16 by job	Sized to each job's data and the host memory
Write layout	repartition(city) before write	A few large files per city instead of many small ones
Broadcast joins	median tables t1 and t2	Turns two shuffle joins into map-side joins
Caching	cleaning table, GBT train/test sets, stream predictions	Each is reused several times; unpersist() frees memory afterwards
Checkpointing	GBT checkpoint directory	Breaks long lineage; an earlier full-scale run died with a StackOverflow at 3 h 50 m

5.5 Feature engineering (FeatureExtraction.scala)

Features are computed per station over time-ordered windows. Rolling windows use rangeBetween on the timestamp value rather than on row position, so a missing hour shortens the window instead of silently reaching further back in time; this makes the rolling features gap-safe.

```
val dedupW = Window.partitionBy("station_id","variation_bucket","ts_unix")
                   .orderBy(col("pm25").desc_nulls_last)
df = df.withColumn("_rn", row_number().over(dedupW)).filter(col("_rn")===1)

def hoursBack(n: Int) = Window.partitionBy("station_id","variation_bucket")
  .orderBy("ts_unix").rangeBetween(-(n-1).toLong * Hour, 0L)   // value-based, gap-
safe
df = df.withColumn(s"${p}_avg${h}h", avg(col(p)).over(hoursBack(h)))
```

The feature groups are: the current pollutants; history (lag_1h, rolling 3/6/24 h means, std_6h, delta_1h, surge_ratio); CPCB sub-indices and rolling AQI (24 h averages for PM, NO_2, SO_2, NH_3 and 8 h for CO and O_3); weather; time of day as sin/cos; traffic and urbanization context; and the provenance flags.

The label is the next-hour $PM_{2.5}$, defined only when the next record is exactly one hour later (next_ts = ts + 3600 s). An earlier positional lead() would have mislabelled 8.69 % of targets across gaps; 12.3 % of rows have no genuine next hour and are dropped rather than guessed.

5.6 The forecasting model

The forecaster is Spark MLlib's GBTRegressor. Gradient boosting builds one small tree at a time; each new tree is fitted to the errors (residuals) the ensemble still makes, and its output is added scaled by a small step size, so the forecast is the sum of the trees. It captures non-linear interactions (wind, temperature and humidity together), needs no feature scaling, copes with mixed units and one-hot flags, is native to MLlib so training distributes across the data partitions, and yields feature importance.

Table 8. GBT configuration

Setting	Value	Why
Trees (maxIter)	300	Small steps need many rounds; the plan's default of 20 was far too few
Depth (maxDepth)	6	Up to 64 leaves per tree
Step size (η)	0.08	Gradual correction of errors
Row subsampling	0.8	Decorrelates the trees
Feature subset	sqrt (≈ 9 of 86)	A random handful of features per split
Max bins	64	Histogram split search: speed and memory
Target	$PM_{2.5}(t+1) - PM_{2.5}(t)$	Predict the change, then add the current value back
Training rows	10,000,552	Seeded 13.41 % sample of 74,544,984 (RAM ceiling)
Test rows	18,671,121	Not sampled: every row of the hold-out is scored

Delta target. Trees are piecewise-constant: asked for the level they spend capacity re-deriving “about the same as now”. Predicting the change removes that burden. In a controlled local reproduction the level target gave R^2 0.9098 (5.6 % better than persistence) and the change target R^2 0.9201 (11.3 % better) with the same trees and features.

Chronological split. A random split leaks on a time series: consecutive hours correlate at about 0.81, so hour T in the training set and T+1 in the test set means the model has effectively seen the answer. Training uses the earlier part of the history; the test window is 2022-07-01 to 2023-03-31, the most recent.

Persistence gate. Every run computes the “next hour = this hour” baseline on the same test rows and prints *** FAIL *** if the model does not beat it, so a model can never be reported without that comparison.

5.7 Live enrichment (LiveEnrichment.scala)

Raw CAMS JSON lacks 26 of the model's 54 numeric features and the provenance flags. One Spark job on YARN joins CAMS, the weather feed and the city profiles: eight weather features come from the Open-Meteo weather API, five derived features (ventilation, scrub factor, stagnation, temperature×humidity, is_raining) are computed once weather exists, three context features come from city_profiles.csv, and ten *_was_imputed flags record whether the API returned a value. Two unit conversions were measured, not assumed: CO from µg/m³ to mg/m³ (divide by 1000; unconverted, structured CO averages 0.95 against 566 for CAMS) and pressure from hPa to mmHg (× 0.750062; the result 753.81 matches the structured median of 753).

The October run joined 82,800 rows (230 cities × 360 hours, 2026-10-01 to 2026-10-15) with no nulls in the model-critical columns.

5.8 Structured Streaming speed layer (AirQualityStream.scala)

AirQualityStream watches an HDFS directory with the file source and a 10-second trigger. A micro-batch alone cannot see a station's previous 24 hours, so each batch is unioned with a persisted 30-hour history table on HDFS, features are computed over the union, predictions are emitted only for the new rows, and the history is then updated. A persisted table was chosen over in-memory state because it survives job restarts. The GBT pipeline and model are loaded from HDFS (MLlib's on-disk format is language-neutral).

```
val raw = spark.readStream.schema(InputSchema).option("header","true")
  .option("maxFilesPerTrigger","1").csv(StreamInput)        // HDFS file source
raw.writeStream.option("checkpointLocation", CheckpointDir)
  .foreachBatch { (batch: DataFrame, id: Long) =>
     // union with 30 h history -> features -> GBT -> AQI ->
alerts.write.partitionBy("city")
  }.trigger(Trigger.ProcessingTime("10 seconds")).start()
```

5.9 Alert engine and delivery

The forecast $PM_{2.5}$ is converted to a CPCB National AQI (the maximum sub-index over the pollutants) with the band and the dominant pollutant, and a warning is written when the forecast AQI exceeds 150. Delivery is a separate step (send_alerts.py) that picks, per city, the worst forecast hour in the next six hours and applies de-duplication: a city is messaged only if it is newly under warning, if its severity got worse, or after a six-hour cooldown while still warned. When a city leaves warning its state is cleared, so a later recurrence alerts again. Delivery channels are Telegram (people subscribe by messaging the bot “/city Hisar”) and e-mail, with an always-on outbox log. Without credentials it runs as a dry run and never claims a message reached a phone.

An hourly loop (run_hourly_cycle.py) separates two cadences: scoring (fetch, enrich, stream, export) every six hours, because the CAMS source refreshes only a few times a day, and dispatch every hour from the already-scored forecast. Only forecast hours newer than the last scored hour are handed to the stream. A replay of the real October forecast showed the rule working: at 20:00 two cities were messaged, at 21:00 those two were suppressed while two others were messaged, and the next morning no city was under warning.

5.10 Hive serving layer

Three external tables (unified_airquality, clean_airquality, stream_alerts) and three views are defined over the Parquet already in HDFS, exposed through HiveServer2. Because the tables are external they add SQL over 154 million rows at no storage cost and dropping a table removes only metadata. All six demonstration queries return real results; one of them shows Delhi's seasonal cycle (Section 6.6).

5.11 Public warning board

A single static HTML page is built from a Hive export: Hive tables are queried by an export job into one JSON, which is embedded in the page. It opens without a server or internet access. Each warning card shows the message residents would receive (forecast AQI, the hour, the cause, CPCB health advice and what to do), with search, a severity filter, a map, warnings per hour, a WhatsApp share button and an “Alert me” browser-notification panel.

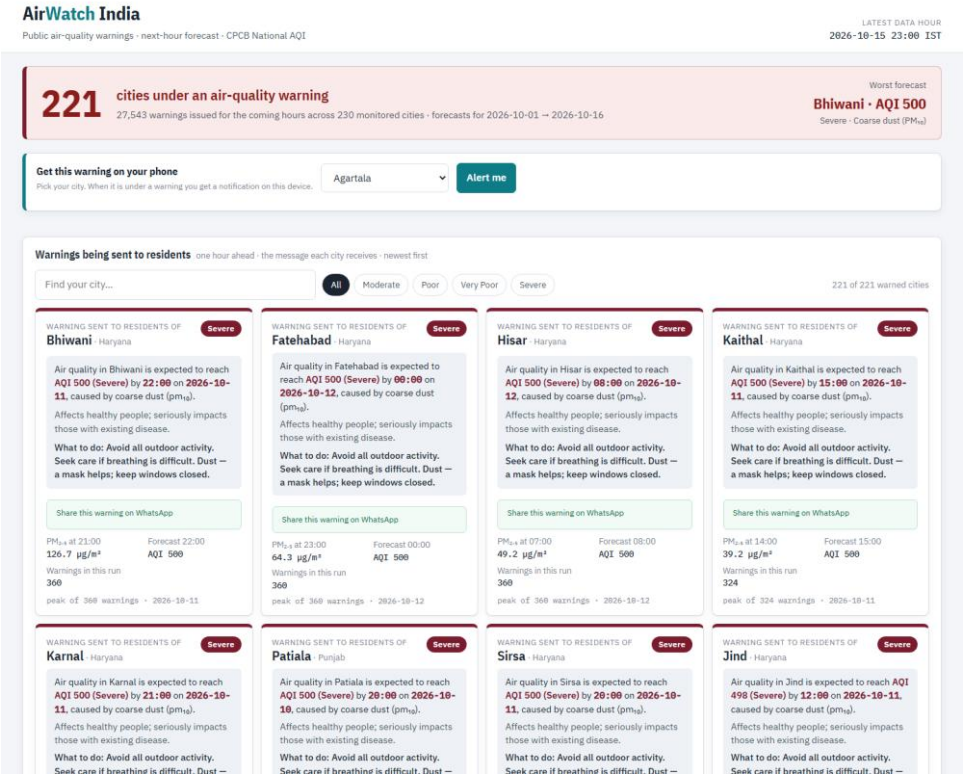


Figure 9. Public warning board (October 1–15, 2026 run): 221 cities under warning

6. Results

6.1 Forecast accuracy

The headline figure is quoted on real rows: the synthetic rows are easier to predict, so the blended number would flatter the model. On 2,074,569 real station-hours the GBT reaches R^2 0.8910 against 0.8587 for persistence, and cuts RMSE from 24.831 to 21.806 µg/m³ (12.18 %). On all 18,671,121 test rows the figures are R^2 0.8884 against 0.8563 (RMSE 22.429 against 25.443, 11.85 % lower). Persistence is an honest bar: doing nothing already scores 0.8587.

Table 9. Evaluation results

Test rows	Model	R^2	RMSE	MAE
Real (n = 2,074,569)	Persistence baseline	0.8587	24.831	9.937
Real (n = 2,074,569)	**GBT forecaster**	**0.8910**	**21.806**	**8.926**
All (n = 18,671,121)	Persistence baseline	0.8563	25.443	10.357
All (n = 18,671,121)	GBT forecaster	0.8884	22.429	9.353

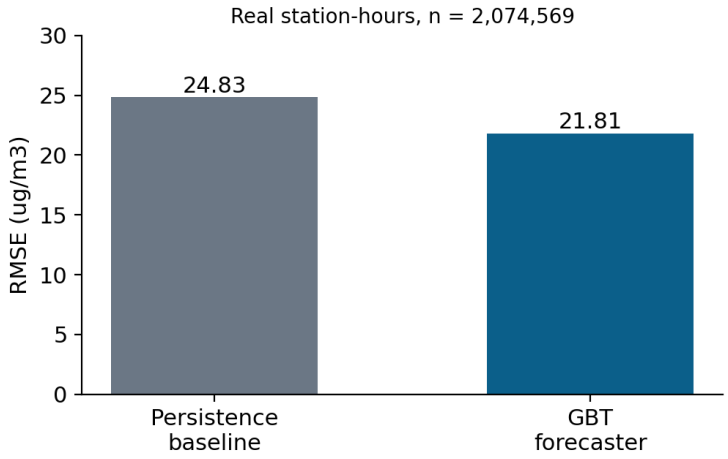


Figure 10. RMSE of the persistence baseline and the GBT on real rows

6.2 Model comparison and tuning

Tuning was done by controlled experiments rather than a blind grid search: tree count (20, 120, 300) and level versus delta target. A pilot on 19 cities with 120 trees gave a 5.67 % RMSE reduction; scaling to all 241 cities and 300 trees roughly doubled it to 12.18 %.

Table 10. Experiments and RMSE reduction against persistence

Experiment	RMSE reduction
Persistence (next hour = this hour)	baseline
Pilot: 19 cities, 120 trees	5.67 %
Local reproduction: level target	5.6 %
Local reproduction: delta target	11.3 %
Full: 241 cities, 300 trees, delta target	**12.18 %**

6.3 What the model learned

Where a station is now (pm25) and which way it is moving (pm25_delta_1h, pm25_surge_ratio) matter most. The pm25_was_imputed provenance flag ranks sixth of 86 inputs: the model learned to discount filled-in readings, direct evidence that flagging every imputed value was the right design.

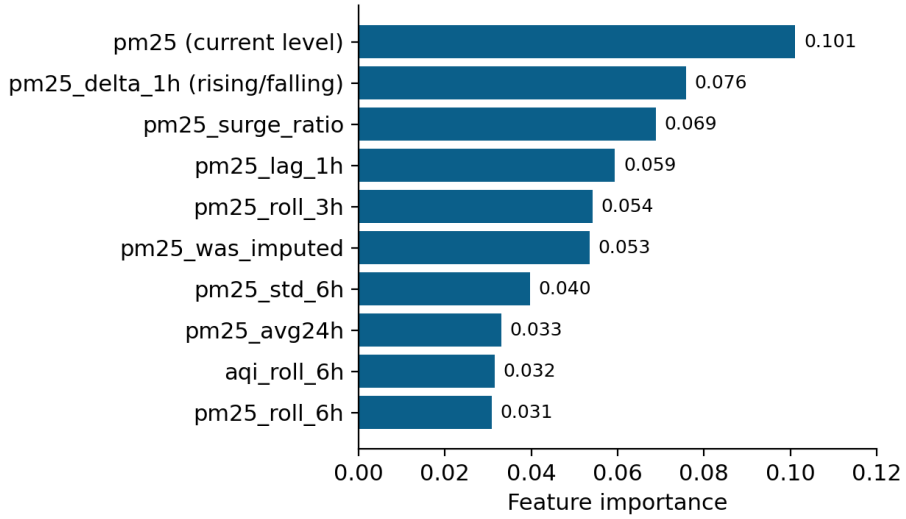


Figure 11. Top-10 feature importances of the trained GBT

6.4 Streaming performance

After the JVM warms up, 120-row replay batches take about 7 seconds end to end (10.8, 7.8, 7.0 and 6.9 s for batches 0–3). The October batch scored 82,800 rows in one pass in 57.8 seconds with no history (a cold start).

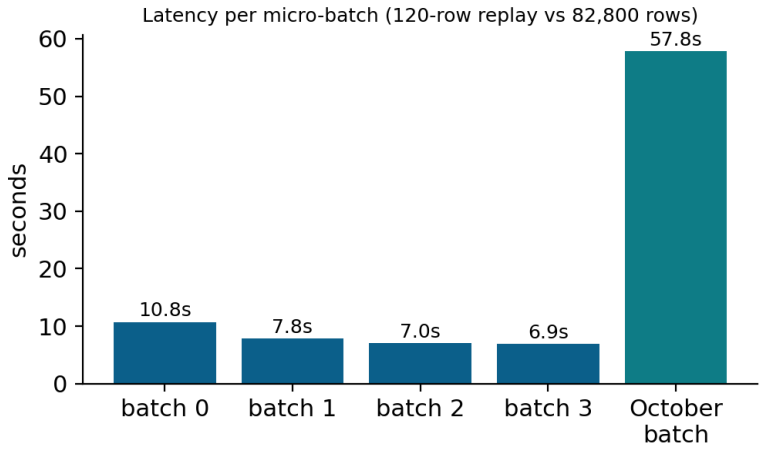


Figure 12. Latency per micro-batch

6.5 October 2026 live run

The October 1–15 run scored 82,800 live rows and raised 27,543 warnings across 221 of the 230 monitored cities; 32 cities were forecast Severe (AQI above 400). The worst forecasts were in Haryana and Punjab (Hisar, Bhiwani, Fatehabad, Kaithal, Karnal, Patiala, all AQI 500 driven by coarse dust PM_{10}), the post-monsoon dust and stubble-burning belt. The cause mix is 44 % ozone and 36 % coarse dust, which implies different advice: avoid midday exertion for ozone, wear a mask for dust.

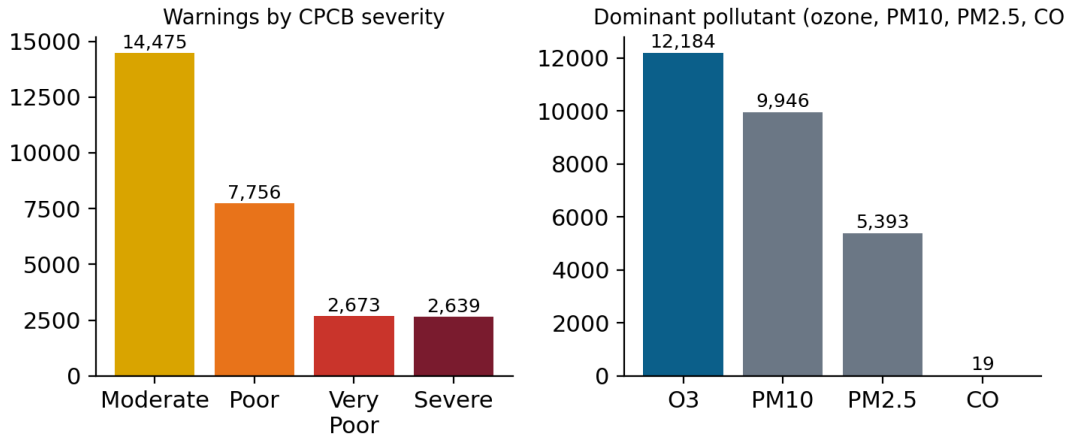


Figure 13. October 2026 warnings by CPCB severity and by dominant pollutant

6.6 Seasonality

Delhi's measured $PM_{2.5}$ swings 5.5 times between the August monsoon low (38.8 µg/m³) and the November peak (213.6 µg/m³). This is why thirteen years of history are kept: a model trained on one season would not generalise.

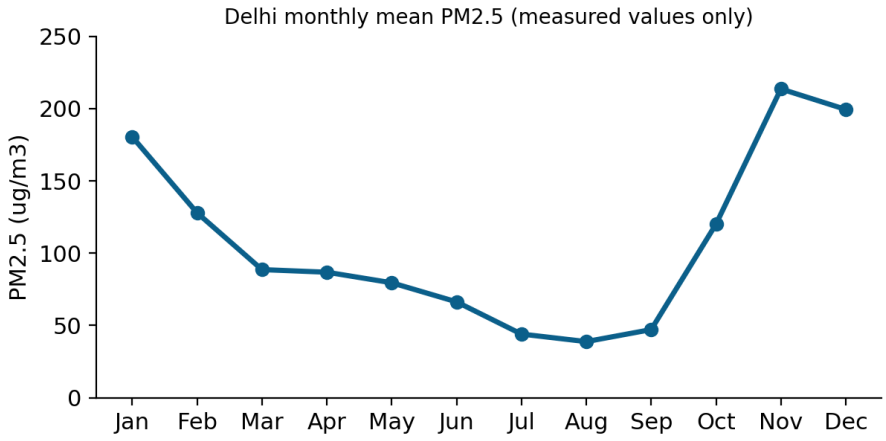


Figure 14. Delhi monthly mean PM2.5, measured values only (Hive query)

6.7 Business insight

- A warning reaches residents one hour ahead, with CPCB health advice and the cause.
- Alerts can be targeted: the Haryana–Punjab dust belt needs a different message from the ozone-driven warnings elsewhere.
- Because every filled value is flagged and accuracy is quoted on real rows, the figures can be defended.

7. Validation and Testing

7.1 Defects found by running the code

Eleven defects were found, four in the data and seven in code that had been written but never run. Two gave plausible wrong answers with no error at all, which is more dangerous than a crash: 16,560 live rows went into a stage and zero came out, and a Hive table returned correct counts next to NULL timestamps. The habit that caught them was to reconcile every output count against an independently known number.

Table 11. The eleven defects

Defect	How it was caught / fix
Flags not binary (24,725 distinct values)	Round to 0 / 1
T+1 label via lead(): 8.69 % wrong horizon	Require next_ts = ts + 3600 s
Impossible values (humidity 104.999 %)	Clamp to physical range
Only 43 % of rows have all 7 pollutants	Three-tier imputation, every fill flagged
Substring join double-counted Mumbai / Noida	44,544 ≠ 44,160 rows; exact-basename join
Streaming used ts_unix before it existed	Alerts printed then the stream stalled; derive ts_unix once
VectorAssembler silently dropped every live row	**16,560 in, 0 out; fill after flagging provenance**
paths.py built hdfs:////host (four slashes)	Printed the resolved path; fixed the join
HDFS write died: no spare DataNode	Replication 2 for derived layers, replace-datanode policy NEVER, 180 s timeouts
numpy missing in the Spark image	Training failed at import; installed
Hive table referenced a column that does not exist	**NULL timestamps with correct counts; table fixed**

7.2 Independent cross-validation against CAMS

The CPCB station data was compared with CAMS on 1,152,707 matched city-hours. Particulates track moderately ($PM_{2.5}$ r = 0.565, PM_{10} r = 0.558), ozone and NO_2 less (0.361, 0.219) and SO_2 shows essentially no agreement (0.083). The result is mixed and is reported as mixed. A synthetic-versus-CAMS comparison was also run and discarded: city-hour averaging cancels the synthetic perturbation, so the test cannot tell synthetic from real.

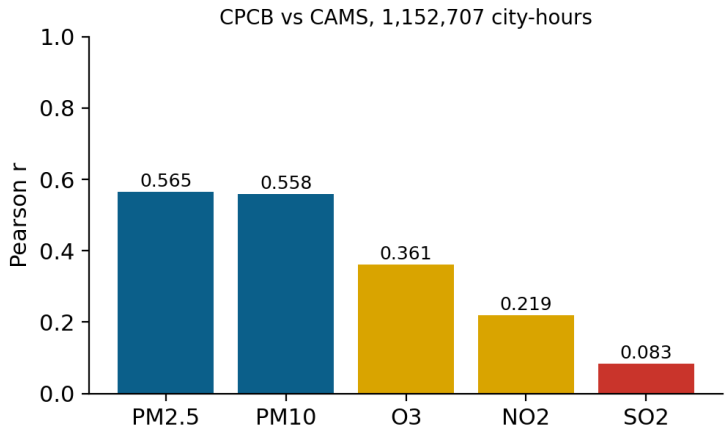


Figure 15. Pearson correlation between CPCB and CAMS by pollutant

8. Evidence

Every claim traces to a preserved log in docs/run_logs, a screenshot of the running cluster, or a script that reproduces it. The table below maps the key claims to their proof.

Table 12. Claim to evidence

Claim	Where the proof is
154,583,979 rows, one schema	spark_on_yarn_demo.log; merge_run.log
4 DataNodes, replication 3, HEALTHY	dfsadmin and fsck screenshots (Section 5.1)
Spark job on YARN reading HDFS	YARN UI screenshots; spark_on_yarn_demo.log
146,276,379 rows cleaned, 241 city partitions	CleanSelect run and HDFS browse (Section 5.3)
93,216,105 labelled rows, 86 features, 73.5 min	phase4_fullscale.log
R^2 0.8910 vs 0.8587, RMSE − 12.18 %	trackB_train_fullscale.summary.log
CPCB vs CAMS cross-validation	crossval.log; scripts/04_cross_validate_overlap.py
October 2026: 82,800 rows, 27,543 alerts, 221 cities	october_live_cycle.log; october_stream_scoring.log
De-duplicated alert dispatch	hourly_dedupe_demo.log; scripts/send_alerts.py
Hive tables and the Delhi seasonal query	hive_create.log; hive_queries.log
Every number reproducible	scripts/_audit (15 scripts); docs/WORK_LOG.md

Verbatim evaluation block of the 300-tree training run (trackB_train_fullscale.summary.log):

```
train 10,000,552 rows | test 18,671,121 rows
test window: 2022-07-01 -> 2023-03-31
PERSISTENCE BASELINE  R2=0.8563 RMSE=25.443
trained on 10,000,552 rows, 300 trees
REAL ROWS ONLY (n=2,074,569)
  persistence baseline  0.8587  24.831  9.937
  GBT model             0.8910  21.806  8.926
  RMSE reduction        12.18%
PASS (real rows): beats persistence by 0.0323 R2
```

Phase 4 summary (phase4_fullscale.log) and the October streaming batch (october_stream_scoring.log):

```
rows in            : 146,276,379
rows after dedupe  : 128,660,967
rows with T+1 label: 93,216,105
FEATURE VECTOR DIM : 86      partitioned by: city      elapsed: 4411.0 s

[batch 0] no history yet - first run
[batch 0] *** 27543 ALERTS ***
[batch 0] rows in=82,800  predicted=82,800  alerts=27,543  latency=57,820 ms
```

9. Challenges and Limitations

Table 13. Challenges, causes and resolutions

Challenge	Root cause	Resolution
Spark could not write to HDFS from the host	The NameNode returns container-internal DataNode addresses	Run Spark inside the cluster network
Full-scale write died after computing	DataNodes unresponsive under memory pressure; no spare node at replication 3	Driver 7 → 5 GB, replication 2, replace-datanode policy NEVER, 180 s timeouts
YARN executors killed (exit 137)	4 × 3 GB NodeManagers advertise 12 GB on a 15.7 GB host	Phase 4 in local mode; YARN proven by the enrichment job
Shuffle spill filled the C: drive	Scratch volume lives in the WSL2 disk (peaked at 65 GB)	Guard job kills below 30 GB free; never tripped
Live rows silently dropped	VectorAssembler skip and an always-null CAMS ammonia column	Fill after flagging provenance
Live feeds shared zero hours	Fetched weeks apart	Fetch both in one cycle; window report added
Stale weather reused in a refresh	Resumable fetcher skipped existing files	Staging folders are cleared before each fetch

9.1 Limitations

- 88.9 % of the structured rows are synthetic. Results are therefore reported on real rows only.
- The model was trained on a seeded 10 M-row sample of 74.5 M rows because of single-host RAM; evaluation is not sampled.
- CAMS is a model product, not a sensor feed, and refreshes only a few times a day; SO_2 agreement with CPCB is poor (r = 0.083). Real-time CPCB station data needs an API key.
- The October result is a snapshot of one run scored as a single batch with a cold-start history; the page does not update itself.
- Telegram and e-mail delivery are built and dry-run tested but no live message was sent (no bot token configured); SMS needs a paid gateway. The full scoring stage of the hourly loop was run end to end, but the hand-off to the stream was not exercised because there were no new forecast hours.
- 230 of 241 cities have live coverage.

10. Conclusion and Future Work

The platform does what its title says. A fault-tolerant four-node HDFS lakehouse holds 154.6 million rows; Spark on YARN runs the Scala cleaning and feature jobs and a 300-tree GBT forecaster that is 12.18 % more accurate than the honest baseline on real rows; Structured Streaming scores live data in ten-second micro-batches; and the alert engine turned an October run into 27,543 warnings with health advice, served through Hive, a public warning board and a de-duplicated delivery loop. The contribution is not only a model but a measured, reproducible and honest pipeline: every filled value is flagged, accuracy is quoted on real rows, and eleven defects were found by running the code.

10.1 Future work

- Schedule the hourly loop on a small always-on server and connect a paid SMS gateway; scale Telegram sign-up.
- Compare against Random Forest and XGBoost baselines.
- Add neighbour-station features (a GraphX station network).
- Resolve the 11 unmatched localities and add ground-truth SO_2 validation.
- Move to a multi-host cluster so the full 74.5 M training rows can be used, and use a real-time CPCB feed with an API key.

11. Repository and How to Reproduce

The code, logs and this report are in the GitHub repository https://github.com/jaganath010/A-Scalable-Big-Data-Platform-for-Real-Time-Urban-Air-Quality-Tracking-and-Public-Safety-Alerting. The public warning board is at https://jaganath010.github.io/A-Scalable-Big-Data-Platform-for-Real-Time-Urban-Air-Quality-Tracking-and-Public-Safety-Alerting/ (GitHub Pages).

Table 14. Repository layout

Path	Contents
scala/streaming/src/main/scala/	CleanSelect, FeatureExtraction, LiveEnrichment, AirQualityStream (1,536 lines of Scala)
scripts/	PySpark: 03_train_gbt.py, data fetchers, merge, cross-validation, run_daily_live_cycle.py, run_hourly_cycle.py, send_alerts.py, _audit/
hive/	External tables, views, demonstration queries
hadoop/	Docker Compose for HDFS, YARN and Spark (11 containers)
dashboard/	Hive export, build script and the static warning board
demo/	Spark-on-YARN demonstration (start_demo.ps1)
presentation/	Final review deck and the first-review deck
docs/	This report, WORK_LOG.md, ARCHITECTURE.md, ALERTING.md, dataset report, run_logs/

11.1 Running it

1. Start Docker Desktop and the cluster: docker compose -f docker-compose.yml -f docker-compose.yarn.yml -f docker-compose.spark.yml up -d (in hadoop/).
2. Run the demonstration: powershell -ExecutionPolicy Bypass -File demo\start_demo.ps1 (Spark on YARN over HDFS, then the website).
3. Run the pipeline phases in the order given in RUNBOOK.md (clean, features, train, stream, Hive, dashboard).
4. Alerting: python scripts\run_hourly_cycle.py --once dispatch (dry run until TELEGRAM_BOT_TOKEN is set).

The ~33 GB dataset is not stored in the repository; docs/DATASET_REPORT.pdf describes how it is built and scripts/00*.py regenerate it.

References

[1] Air Quality Prediction using Multi-Source Remote Sensing Data Integration with Hybrid Deep Learning Framework (MAST-Net). Nature, Scientific Reports, December 2025.

[2] Apache Hadoop 3.3.6 Documentation. The Apache Software Foundation. hadoop.apache.org/docs/r3.3.6/

[3] Apache Spark 3.5.6 Documentation (SQL, MLlib, Structured Streaming). The Apache Software Foundation. spark.apache.org/docs/3.5.6/

[4] Central Pollution Control Board. National Air Quality Index — Report of the Expert Group, 2014. Government of India.

[5] Zaharia, M. et al. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. NSDI, 2012.

[6] Shvachko, K., Kuang, H., Radia, S., Chansler, R. The Hadoop Distributed File System. IEEE MSST, 2010.

[7] Friedman, J. H. Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics 29(5), 2001.

[8] Copernicus Atmosphere Monitoring Service (CAMS), ECMWF, via the Open-Meteo Air Quality API (CC-BY 4.0). open-meteo.com

[9] Apache Hive and Apache Parquet Documentation. hive.apache.org, parquet.apache.org